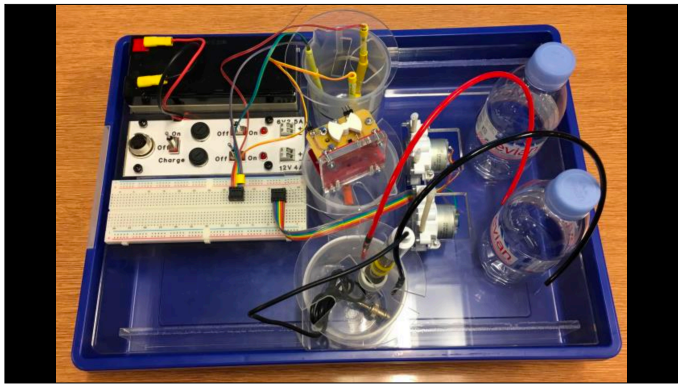
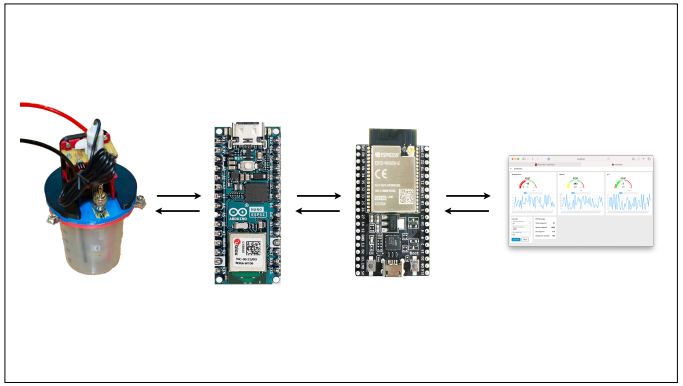
Connectivity

Communication Options for the CS/EEE Bioreactor Challenge

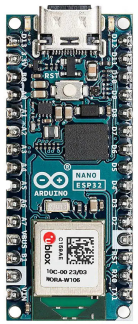
ENGF0001 Engineering Challenges · Autumn 2025



The bioreactor project consists of several subsystems that are managed and coordinated locally by a microcontroller. In addition their behaviour should be able to be monitored and controlled remotely by some kind of dashboard or user interface.



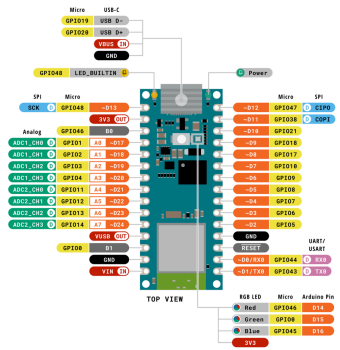
In fact, coordination and communication will be performed using at least two different microcontroller boards.



Arduino Nano ESP32

- **ESP32-S3 MCU**
- **16MB Flash**
- **Analogue & Digital I/O**
- **USB-C**
- **Wi-Fi**
- **Bluetooth**

The Arduino Nano ESP32 is a development board in the popular Arduino Nano format, employing the ESP32-S3 microcontroller (more recent than the ESP32s on the TTGO and DevKitC) and providing an array of analogue and digital input/output pins.

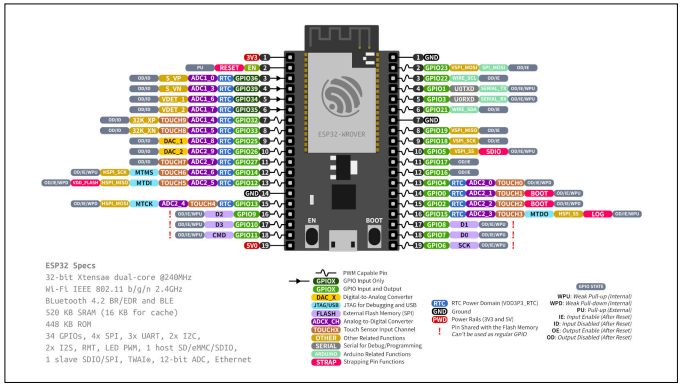


The pins can be configured to perform a variety of functions that will be useful for communicating with the hardware subsystems of the bioreactor, and also to other microcontroller boards.

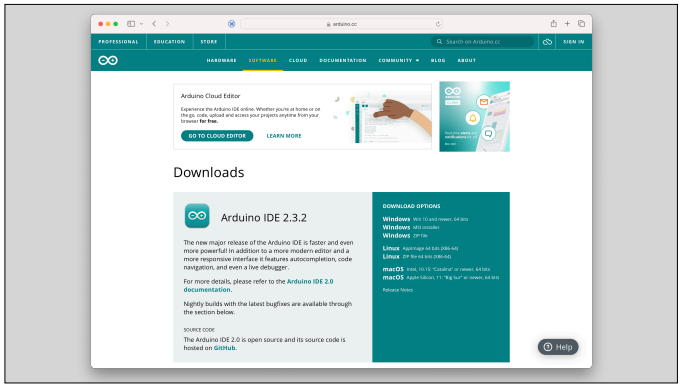
Espressif ESP32 DevKitC V4

- **ESP32-WROVER MCU**
- **4Mb Flash**
- **Analogue & Digital I/O**
- **USB (micro)**
- **WiFi**
- **Bluetooth**

The DevKitC is another microcontroller board with overlapping but distinct capabilities to the Nano.

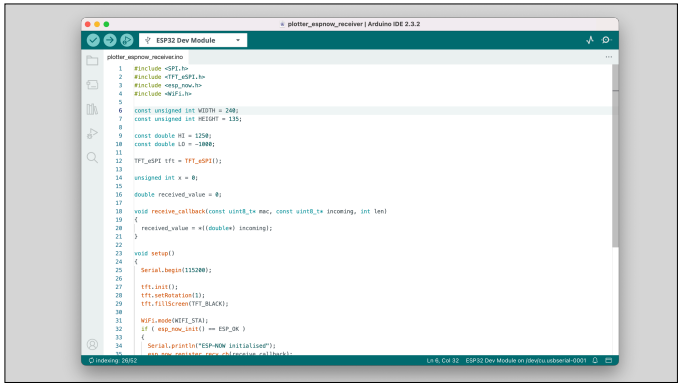


It too has a variety of configurable pins that can be used for direct wired communications. It also comes equipped with WiFi and Bluetooth connectivity out of the box.

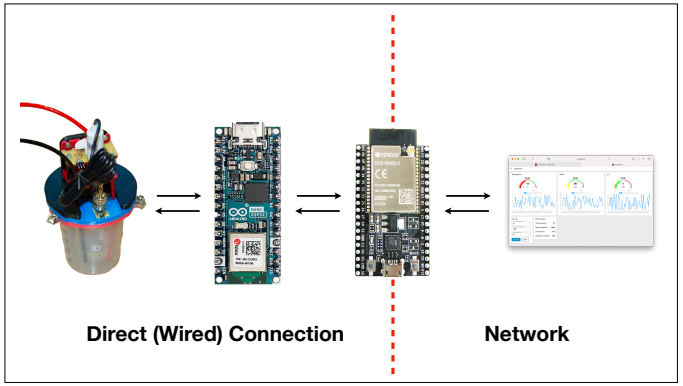


Both boards can be programmed using the Arduino IDE.

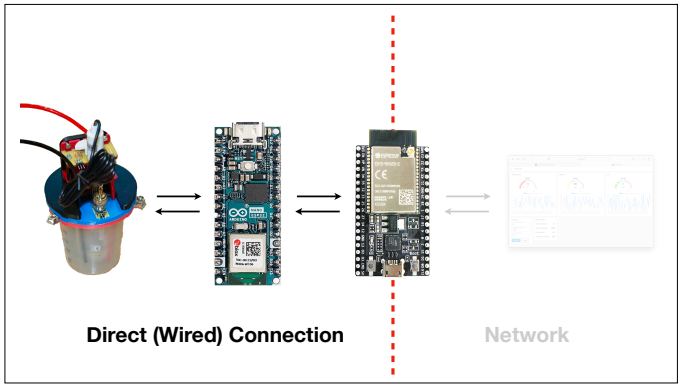
(The Nano ESP32 is supported by default, the DevKitC requires additional board definitions be downloaded and installed. CS students have already done this, so if in doubt talk to your teammates.)



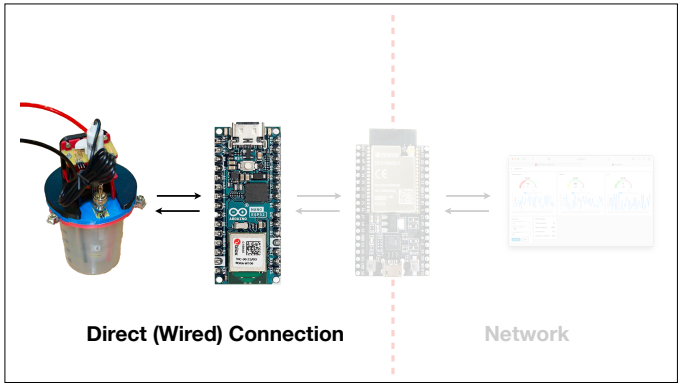
Both devices are coded using the Arduino language, which is essentially standard C++ with some relevant libraries and definitions automatically included.



Communication with the bioreactor can only take place via direct wired connections. Communication between the MCUs will most likely also occur this way, though it is possible to set up wireless connections for this. But probably the DevKitC will act as a bridge to the wider networked world.



We'll start by considering the wired connectivity options.

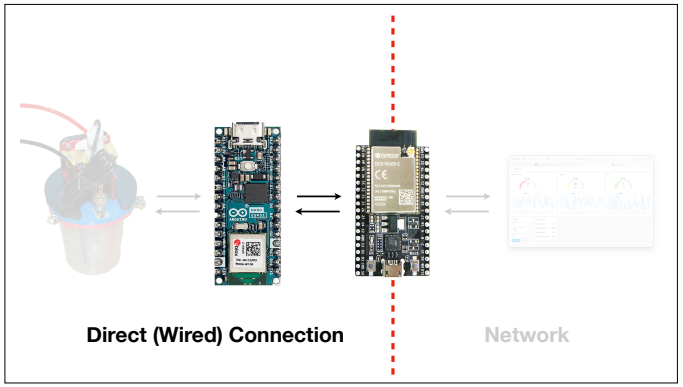


Communication with the individual hardware components of the bioreactor — the sensors and actuators for temperature, pH and stirring — take place via direct analogue or digital signals.

```
int v = analogRead(A0);

pinMode(2, OUTPUT);
digitalWrite(2, HIGH);
```

The physical connections and the format and meaning of the signals will vary from component to component and is up to you to work out. But there are simple functions for reading from and writing to the Arduino's pins.



Communications between the microcontrollers are digital and require more complex structure — **protocols** — to get the information across.

UART

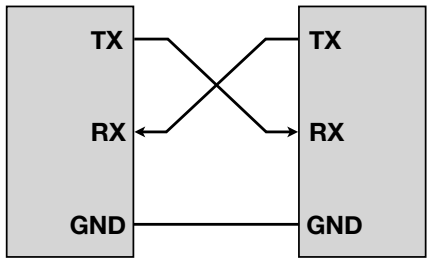
Universal Asynchronous Receiver Transmitter

A common — even ubiquitous — approach to passing digital signals over wires is asynchronous serial or UART. (Strictly, UART refers to the device implementing the protocol rather than the protocol itself, but the terminology gets used pretty loosely.)

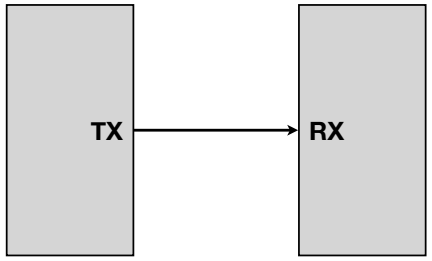
UART

Universal Asynchronous Receiver Transmitter

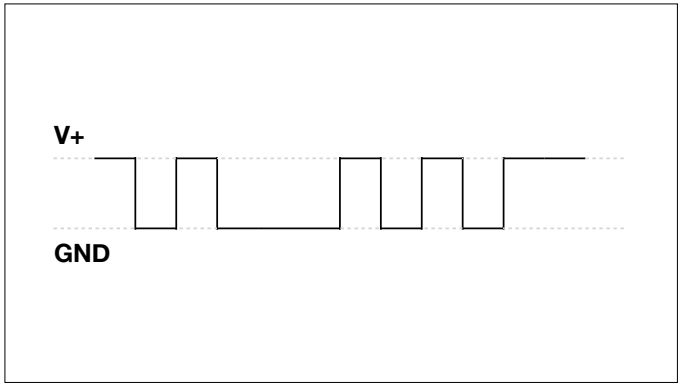
A crucial feature here is that the communications are asynchronous — meaning there is no shared signal keeping the two sides in step with one another, each is responsible for maintaining its own timing.



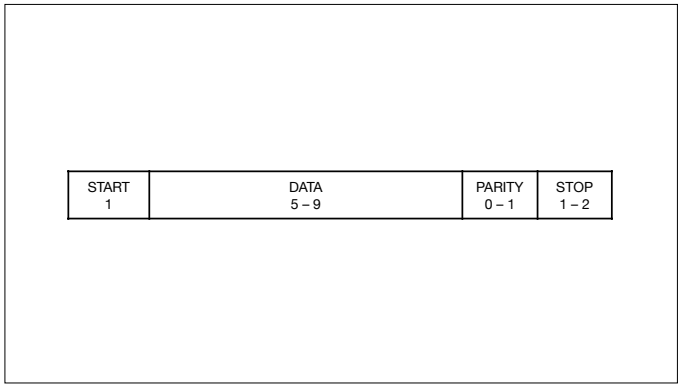
UART is typically used bidirectionally, with a separate wire carrying signals in each direction. Note that in this case the transmitter (TX) on one side must be connected to the receiver (RX) on the other, and vice versa.



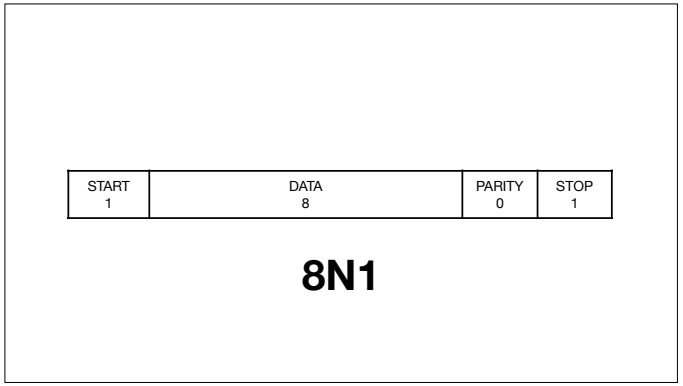
The protocol is the same in each direction, though, so we'll consider only what's going on on a single wire.



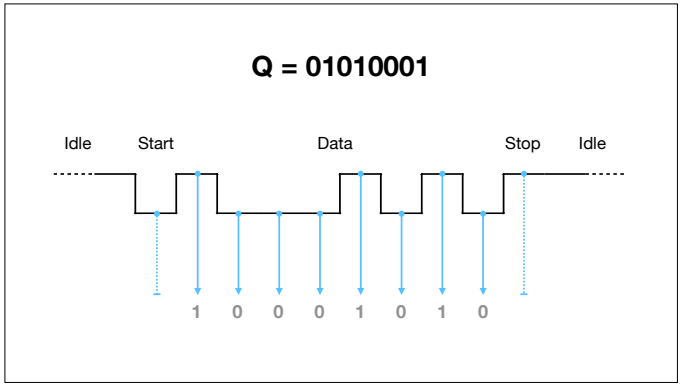
It’s a digital signal, which means the information is carried by the voltage on the wire being in one of two states — which might be called high/low, on/off, true/false, 0/1 etc. Usually low is at ground level and high is above some positive threshold, but the details may vary from device to device. Since the voltage can only be at one level at a time and there are only two recognised levels, only a single bit of data can be conveyed at a time, so data must be passed **serially**, bit by bit.



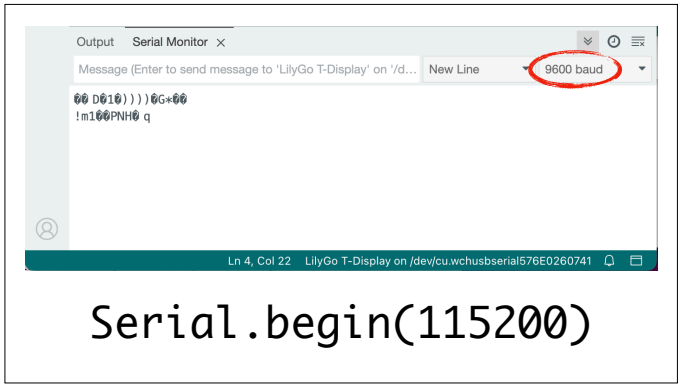
Data is organised into this parade of voltage changes in packets, each containing a little chunk of data. The exact structure is configurable, but a packet always begins with a single **start bit**, alerting the receiver that data is incoming. There is then a sequence of 5-9 **data bits** (the number can vary between devices but is fixed for a particular connection and must be specified externally). There can then be an optional **parity bit**, which provides for a simple form of error checking, but this is rarely used. Finally 1 or 2 **stop bits** declare the packet done.



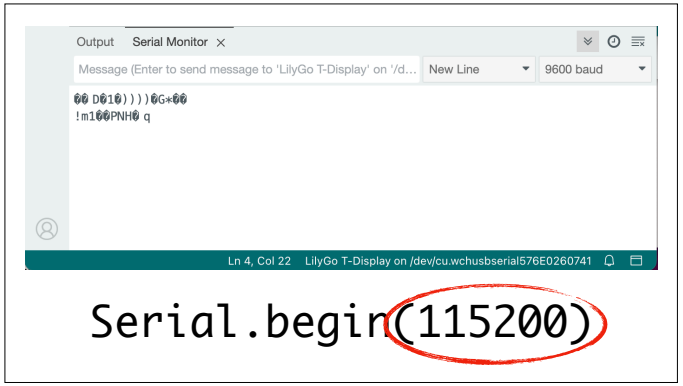
The most common configuration uses 8 data bits, no parity and 1 stop bit, abbreviated as 8N1. Note that the frame bits mean that it takes 10 bits of signal to transfer each byte of data.



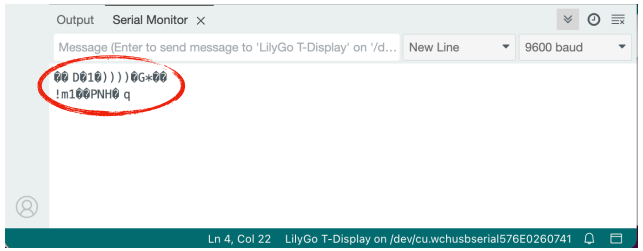
An example of such a packet is illustrated above. (The idle state is conventionally high, though it doesn't have to be.) The sender sets voltages at a fixed interval and the receiver samples those voltages at (hopefully) the same interval, with a small offset. All the timing is relative to the initial start bit signal, but each device is responsible for managing its own interval. The rates on both sides must be set externally.



Most of you will have encountered this in the context of the Arduino IDE, where you have to specify the **baud rate** in the serial monitor window…



…and also set the **baud rate** in the Arduino code.



Serial.begin(115200)

There is nothing forcing these to be the same, it's just incumbent on you to make sure they are. But if they aren't then the data is garbled and nothing useful is communicated.

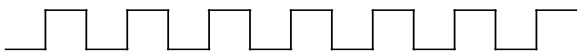
~~#include <Serial.h>~~

https://docs.arduino.cc/learn/communication/uart

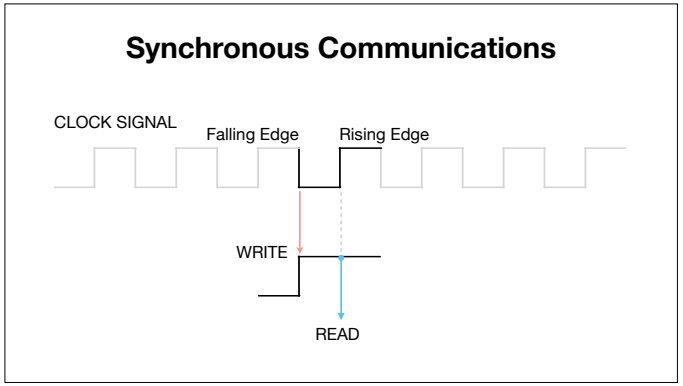
UART communications are available in your Arduino code via the Serial class. This is so ubiquitous that it is not only installed by default but already present in the default code environment so there is nothing to #include.

Synchronous Communications

CLOCK SIGNAL

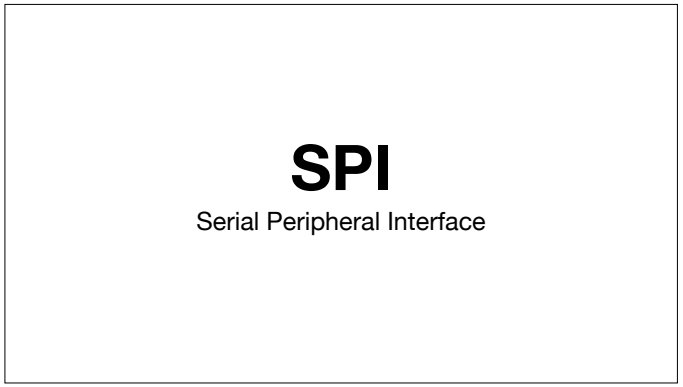


The alternative to asynchronous communication is **synchronous** communication, where there is some shared timing signal to keep both parties in step. This is typically in the form of a simple pulse train or square wave (though probably not quite as perfect as depicted here).

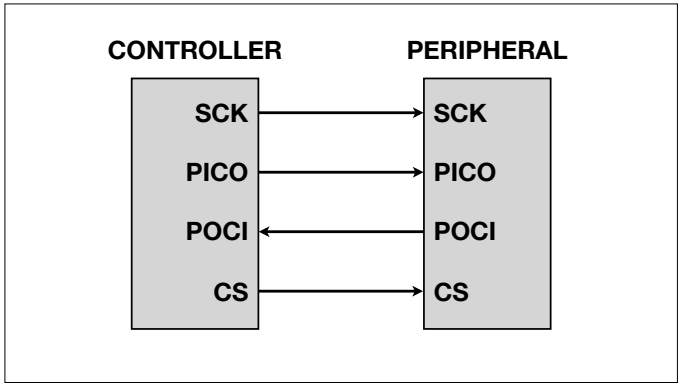


Each cycle of the waveform includes a rising edge and a falling edge. The exact cueing from these may vary between devices and configurations, but typically the sender and receiver will synchronise to different edges. This provides a margin for error in the synchronisation and reduces susceptibility to timing inconsistencies

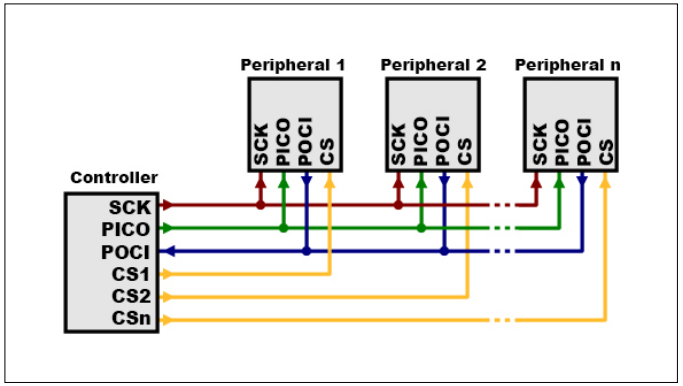
---



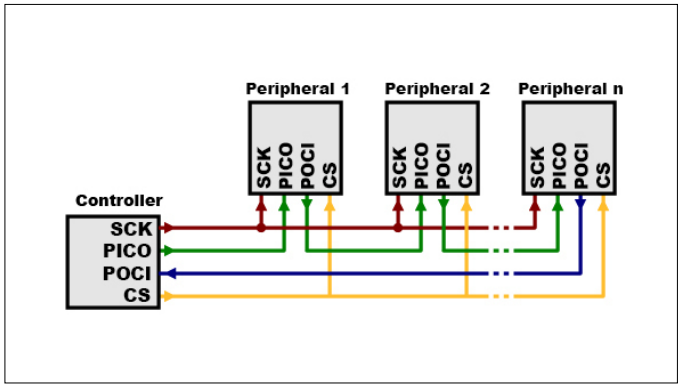
One common synchronous serial communication protocol is SPI.



This uses four wires to manage communication between devices performing two different roles. A controller provides the clock signal and controls the process, while a peripheral sends and receives data on cue. There are dedicated data lines in each direction, labelled in terms of the two roles: peripheral-in/controller-out (PICO) and peripheral-out/controller-in (POCI). PICO connects to PICO on both devices and POCI to POCI, rather than having the TX-RX crossover we saw for UART. A fourth **chip select** (CS) line is used to tell the peripheral it's the one being targeted.



A controller can talk to multiple peripherals independently on shared PICO & POCI lines, but each needs its own chip select line, and the wiring can get out of hand if there are many peripherals.



An alternative **daisy chain** configuration uses only a single CS line for all peripherals. In this case data flows from one device to the next. The controller must send and receive all its device packets together so that each reaches its destination, and must collect all responses similarly, so it can get a bit unwieldy.

```
#include <SPI.h>
```

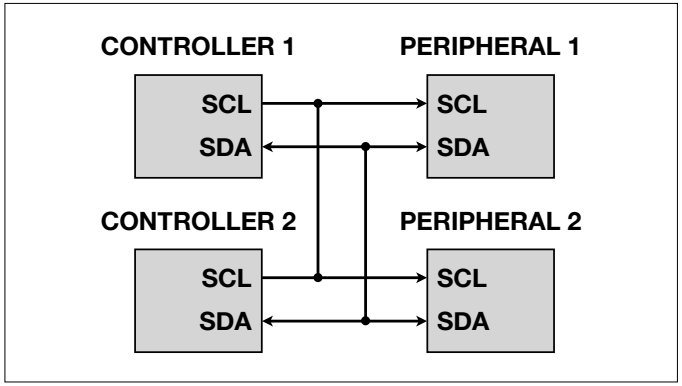
https://docs.arduino.cc/learn/communication/spi

SPI functionality is provided to your Arduino code in the SPI library, which comes preinstalled with your boards but you do need to #include the appropriate header.

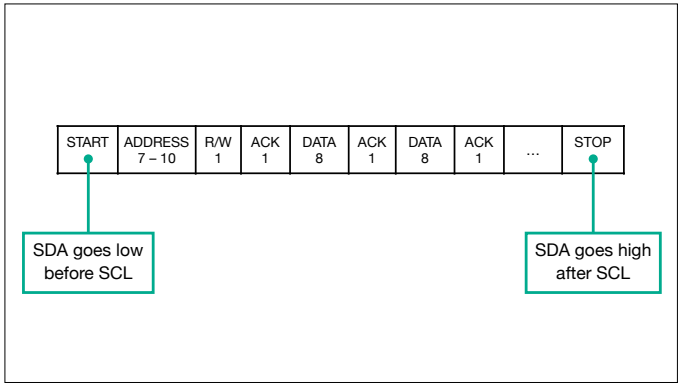
I²C

Inter-Integrated Circuit

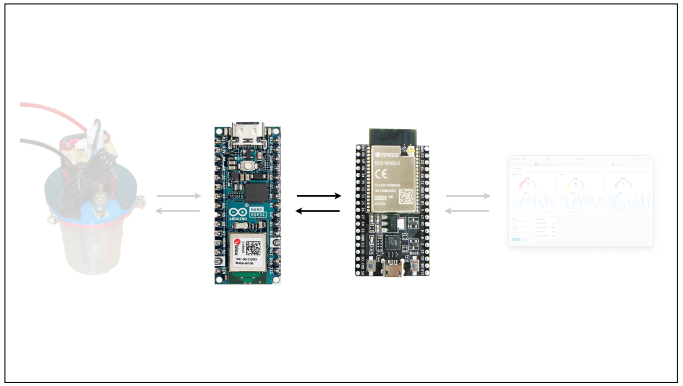
Another synchronous serial protocol for short range wired connectivity is I²C.



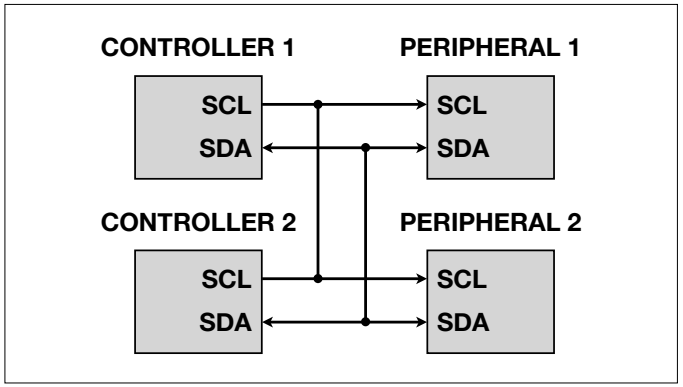
This uses only two wires for communication, and allows many controllers and many peripherals to all share the same two wires.



This wiring simplicity is achieved by using a significantly more complex data link protocol that includes device addressing to direct the data packets to a specific peripheral on the bus. Timing subtleties between the clock and data lines are used to frame the packets.



I²C is a reasonable candidate for communicating between the two microcontrollers.



But note that although multiple controllers can share the bus, only one can be active at a time. Actual data transfer is strictly between one controller and one peripheral. So in order to communicate between microcontrollers, one of them must act as the peripheral.

Controller Reader ← Peripheral Sender

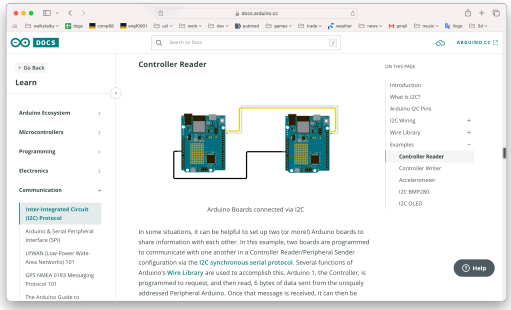
Controller Writer → Peripheral Receiver

There are two ways to organise this, where the controller reads what the peripheral sends, or where the controller writes and the peripheral receives. Both modes can be used at different times.

```
#include <Wire.h>
```

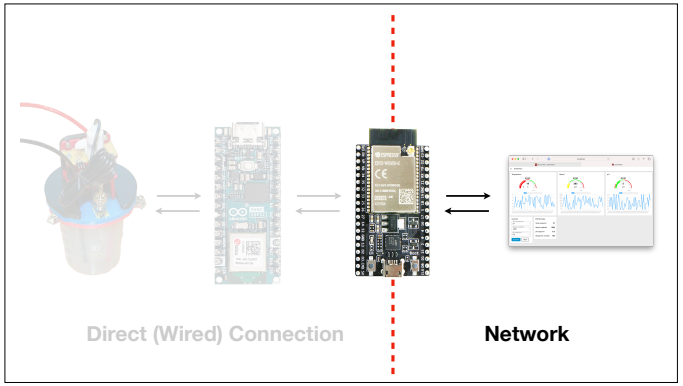
https://docs.arduino.cc/learn/communication/wire

I²C is implemented for Arduino in the Wire library, which again comes preinstalled for your boards but needs a header file to be #included.



https://docs.arduino.cc/learn/communication/wire/#examples

The online documentation includes examples demonstrating how to manage the configurations for controller reader peripheral sender etc.



Now let’s consider the networked side of things, between the second microcontroller and the outside world.

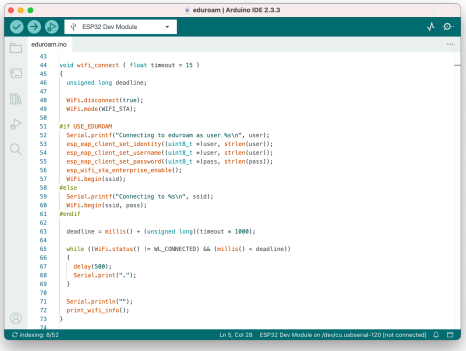
WiFi

As previously noted, the ESP32 provides WiFi support out of the box.

```
#include <WiFi.h>
```

https://docs.espressif.com/projects/arduino-esp32/en/latest/api/wifi.html

The ESP32 WiFi library is installed automatically for your boards, you just need to #include the header. Note that the ESP32 version is maintained separately from the Arduino WiFi library and there may be differences, so use the Espressif documentation as linked here.



Making an ordinary WiFi connection is pretty straightforward, but connecting to eduroam is (slightly) more involved. There is example code on Moodle to demonstrate both.

MQTT

MQTT is a lightweight messaging protocol that is popular for Internet of Things applications.

MQTT

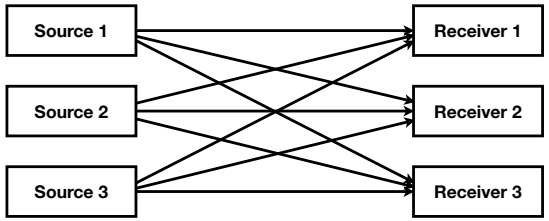
Message Queue Telemetry Transport

The abbreviation doesn't officially stand for anything other than itself, as the original source words are misleading. They are still sometimes used, though.

Publish – Subscribe

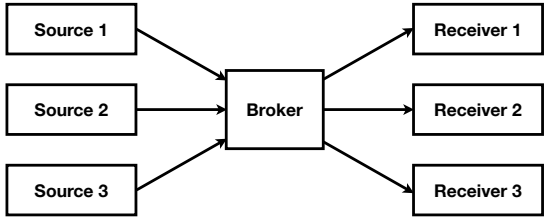
MQTT uses a publish-subscribe model to manage communication between multiple providers of information and multiple consumers of it.

Publish – Subscribe



If the sources and receivers all needed to connect directly to one another the connections would rapidly get out of hand. Doing so would also tightly couple the receivers to the sources, requiring them to know a lot of implementation information that would make the process fragile.

Publish – Subscribe

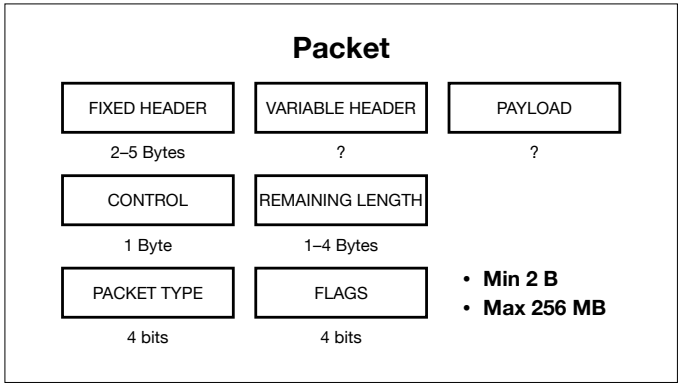


Instead, the publish-subscribe model interposes a broker to mediate the messaging. Clients can publish to the broker and subscribe to messages from the broker without either end needing to know details of the other.

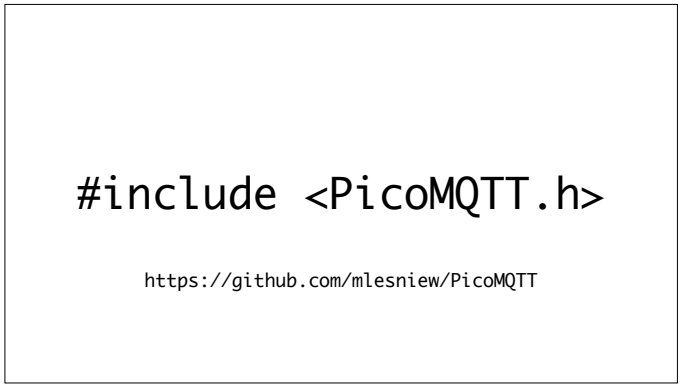
Topics

- UTF8 string labels — `kitchen` `bioreactor` `mars`
- Hierarchical: levels delimited by / — `kitchen/thermostat` `bioreactor/pH`
- Wildcards:
  - + matches a single level — `+/thermostat` `mars/beagle/+`
  - # matches any number of levels — `kitchen/#` `bioreactor/#` `#`
- Nominal max length 65536
  - In practice, brokers & clients may support much less

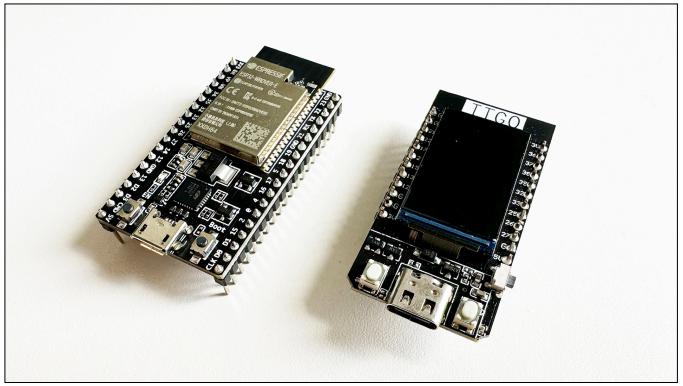
Messages are organised by topic, simple text string descriptors that clients use to declare what they are publishing and/or what they want to subscribe to. The broker routes relevant topics to interested subscribers. Topics are hierarchical and wildcards are supported for subscribing.



MQTT communicates in packets with a variable structure. The smallest valid packet is 2 bytes, and while much larger packets are nominally supported they are rarely used. Most MQTT packets are very lightweight, which is one of the things that makes it popular for resource-constrained IoT applications.



MQTT support is not installed for your boards by default, but there are many implementations available. We recommend PicoMQTT, which you can install via the Arduino IDE's Library Manager, but if you're familiar with something else then feel free to use that instead.



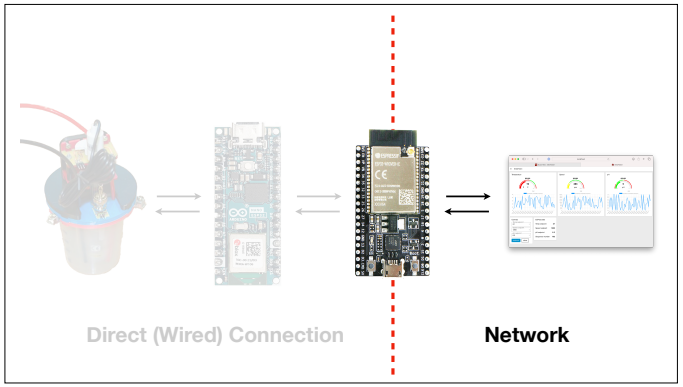
PicoMQTT can run a broker as well as a client, so one possible way to organise your connectivity would be to use one ESP32 to run the broker that the client ESP32 talks to. But note that eduroam won't allow you to run a server on it, so if you take this approach the broker will need to be on some other network you can connect to.

broker.hivemq.com

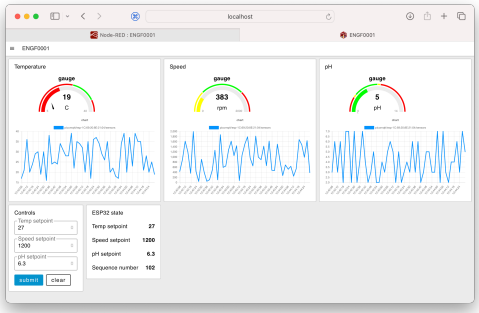
test.mosquitto.org

engf0001.cs.ucl.ac.uk

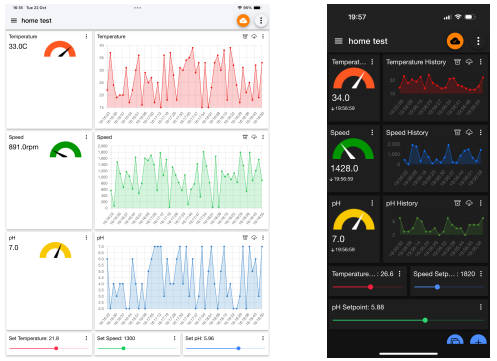
There are a number of free public MQTT brokers on the internet that you can use, and also one running specifically for ENGF0001 on the UCL CS network. But note that the latter is only visible from within UCL, you won't be able to talk to it from elsewhere if you are coding off site.



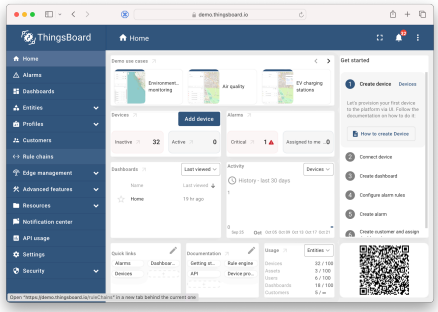
The final piece of the puzzle is the user interface or dashboard that you will use to monitor and control your bioreactor remotely. You are free to do this in any way you please, short of getting someone else to do it for you. No specific software development tools, coding environment or technology are prescribed, you can use whatever you are comfortable with as long as the basic requirements are met: ie, that the interface allows you to monitor the bioreactor parameters over time and set target values for temperature, pH and stirring speed. There are a few weak suggestions on the following slides, but you are welcome to ignore them and do something else entirely.



Node-RED is a "low code" web-based environment that allows you to graphically build web applications that receive, process and display data from disparate sources, known as "flows". The popular FlowFuse Dashboard add-in (https://dashboard.flowfuse.com/getting-started.html) adds features to make it easy to build dashboards.



Various applications are available for iOS and Android to talk to IoT devices using MQTT, and these can be used to construct a control interface if you design your MQTT messaging appropriately.



https://thingsboard.io

Numerous web-based services exist to support connectivity and control amongst disparate IoT devices. One example is ThingsBoard, which offers a commercial service but is also open source and allows you to run your own server if you want. ThingsBoard is versatile and provides a lot of functionality but can be quite complex as a result.

If you're feeling adventurous you could even build your own custom controller around something like the ESP32 LVGL TFT, popularly known as the Cheap Yellow Display, an ESP32 board with a built-in touch sensitive screen.

Any of the above approaches is acceptable, as are many others. Show us what you can do!